

智慧银行实验报告

金融 2032301

202301739

罗玉淇

实验总览

本次系列实验共分四个课时，循序渐进地覆盖了 AI 辅助开发、自动化办公、工程化环境搭建以及知识图谱应用。

课时一：TRAE 环境搭建与贪吃蛇游戏开发（AI 协作基础）

课时二：AI 辅助办公与 Skill 工具应用

课时三：环境完善与 BMAD 初步配置

课时四：项目实施（银行知识图谱构建与问答）

四个实验由浅入深，从单机应用开发到企业级金融智能系统预研，全面锻炼了 AI 工具使用、问题排查和工程化思维能力。

一、课时 1：环境安装与设定

实验一：TRAE 环境搭建与贪吃蛇游戏开发

1、实验目的

安装 TRAE 国内版，熟悉 Builder 模式。

通过自然语言生成贪吃蛇 HTML 游戏，并迭代优化。

掌握 AI 辅助编程的人机协作流程。

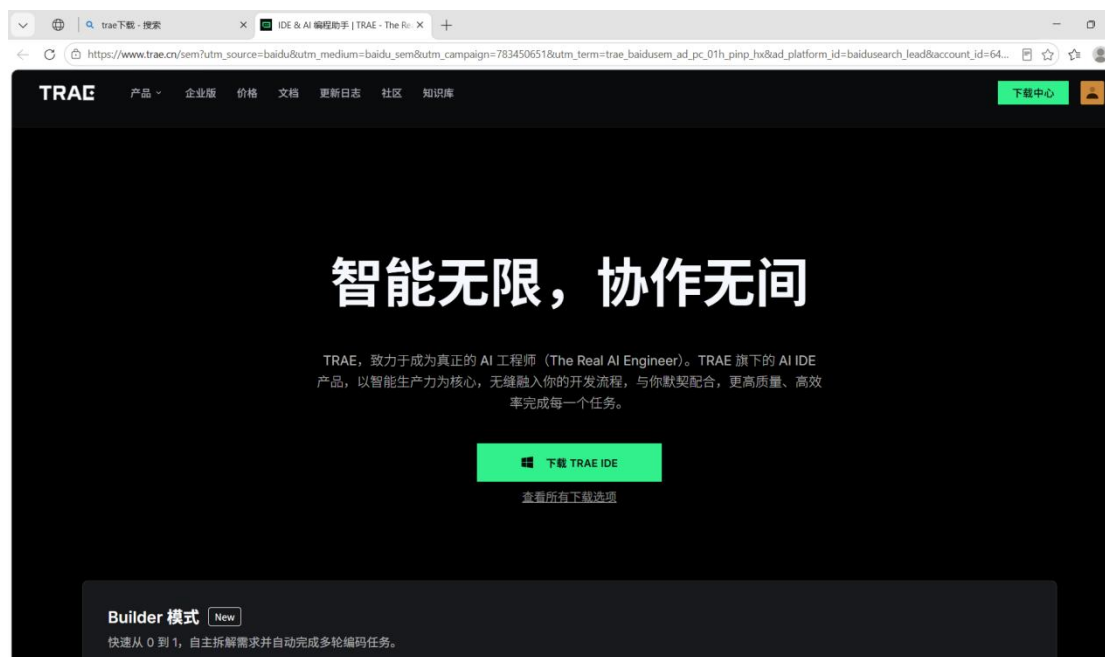
2、环境配置

环境检测结果

检测项	要求版本	当前状态	修复建议
Python	≥3.10	Python 3.11.9	已满足要求
pip	随 Python 安装	pip 24.0	已满足要求
Node.js	≥v20	v24.16.0	已满足要求
npm	随 Node.js 安装	11.13.0	已满足要求
Git	任意版本	2.54.0.windows.1	已满足要求
Flask	最新版本	未安装	<code>pip install flask</code>
pandas	最新版本	未安装	<code>pip install pandas</code>
openpyxl	最新版本	未安装	<code>pip install openpyxl</code>
pypdf	最新版本	未安装	<code>pip install pypdf</code>

3、实验过程

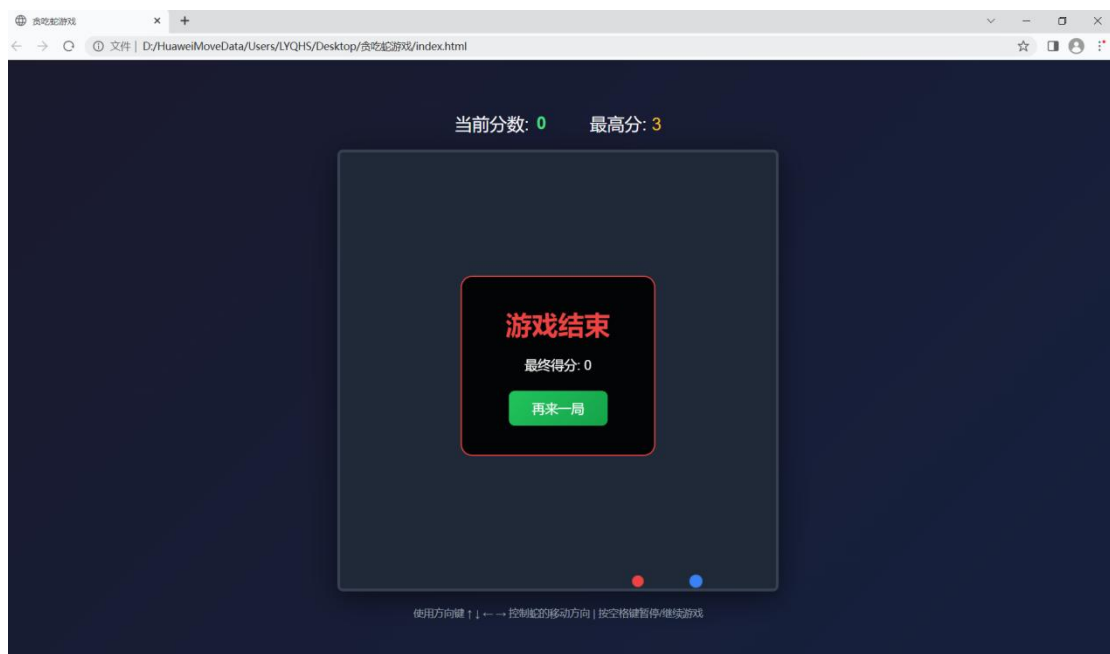
下载 TRAE，登录。



在 Builder 模式下输入游戏需求，生成初始 HTML。



玩自己做的贪吃蛇游戏，然后打出超低分数



4、实验结果

游戏功能完整，功能正常。

掌握了“需求→生成→测试→修正”的协作模式。

5、主要问题及处理

贪吃蛇没有暂停之类的调整，运行过快死的太快没有游玩体验。后面加上了其他设定，让它更好玩一点

↶ 现在游戏看起来有点单调，能不能加个暂停和继续的功能？

6、心得

这次实验让我第一次亲身体验了 AI 辅助开发的完整流程。刚接触 TRAE 时，最大的感受是“不可思议”——用自然语言描述需求，AI 就能在几分钟内生成一个可运行的网页游戏，这种效率远超我的预期。但实际操作下来，我发现 AI 生成代码并非一蹴而就。初次生成的贪吃蛇存在许多问题，需要反复测试、反馈、修正才能逐步完善。这让我认识到，AI 更像一个需要清晰指令的助手，而非全知全能的工具。每次迭代中，我学会了对 AI 输出保持审慎态度，也锻炼了精准描述需求的能力，语言细节直接影响产出质量。

此外，AI 在迭代中覆盖了我之前手工调整的样式，这提醒我：AI 不会主动保留你未明确要求的细节，所有维护意图都需要清晰传达。

总的来说，这次实验让我建立了“快速生成→人工测试→精准反馈→迭代优化”的协作思维，也理解了用好 AI 的关键不在于技术深度，而在于清晰表达与主动把关的能力。。

二、课时 2：简单操作与编写

1、实验目的

注册 CNB 仓库，尝试把东西上传到仓库里面。

安装并使用 docx、pptx 等 Skill，生成办公文档。

体验 AI 在非编程任务中的提效作用。

2、环境准备

TRAE + Node 环境，安装 luban-skill 和 docx 全局包。

3、实验步骤

创建 CNB 仓库并关联本地，执行 git push 遭遇认证失败。

安装 Skill，执行 `skills list` 验证。

调用 pptx 生成 Steam 介绍 PPT

调用 docx 生成客户回访报告（C00001，含封面、目录、页码）。

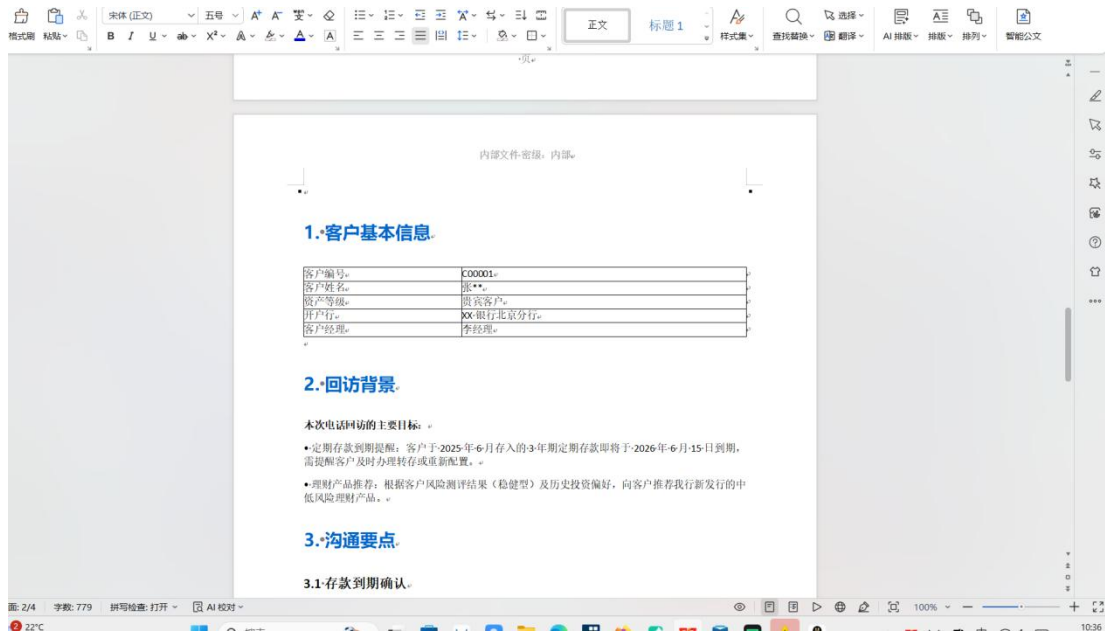
4、实验结果

PPT 和 Word 报告成功生成，格式规范。

TRAE

页码	标题	内容
1	封面	Steam Logo、平台定位、发行商信息
2	什么是 Steam	平台简介、关键数据（30000+游戏、10亿用户）
3	发展历程	2003-2024 年时间线
4	核心功能	游戏商店、自动更新、社区、创意工坊、成就系统、云存储
5	平台优势	安全可靠、价格优势、技术领先、全球覆盖、多平台支持、Steam Deck
6	社交与社区	好友系统、群组、截图分享、评测系统、指南攻略、创意工坊
7	商店特色	促销活动、本地支付、多语言支持、愿望单
8	家庭与分享	家庭共享、家庭监控功能
9	常见问题	下载安装、支付方式、退款政策、客服支持
10	结语	感谢观看、官网链接

设计特色



CNB 推送未成功，不知道是什么原因，但了解了基本命令，准备自己下去修改一下

5、主要问题及处理

按照实验要求做出了一些东西，学会了安装和使用 CNB 并窥探了老师的资料库，然后没找到文件，最后发现是 打开错误

学会创建自己的仓库但是死活没办法上传那个文件，结果发现是电脑配置问题，回去自己用 TRAE 把配置设定完毕了

6、心得

这次实验让我体会到，AI 的价值远不止于写代码，在办公文档处理方面同样潜力巨大。以前做 PPT 或者写一份格式规范的报告，光是调排版、设目录、改页码就要耗费不少时间，现在只需要把需求说清楚，AI 就能自动生成结构完整、格式专业的成品，效率提升非常明显。

但实验中最让我印象深刻的，是 CNB 仓库上传失败的经历。按着步骤一步步操作，最后却卡在 git push 这一步，反复尝试都报错，当时确实有点受挫。后来排查才发现是本地 Git 环境配置不完整，缺少用

户信息和认证凭证。这件事让我明白，工具链是一个整体，任何一环的缺失都会阻断整个流程，AI 再强大也绕不开基础环境的支撑。

另外，AI 生成的 PPT 虽然框架完整，但具体内容比较空泛，比如“热门游戏”那页只有标题和占位符。后来我补充了具体的游戏名称和数据，重新生成后才更丰满。这说明 AI 擅长搭架子，但内容填充和细节把控还得靠人。

总的来说，这次实验让我认识到 AI 办公工具的实用价值，同时也提醒自己：技术工具的上限取决于使用者对整体流程的掌握程度，不能因为 AI 省力就忽视基础配置和人工审核的重要性。

三、课时 3：环境改进

1、实验目的

彻底打通 Git 与 CNB，成功推送代码。

了解 BMAD 方法论，搭建 CRM 项目骨架。

编写 PRD 文档，为敏捷开发做准备。

2、实验步骤

改进实验环境，安装 GIT 以及一系列相关软件，终于把上节课要求上传的东西上传到了 CNB 里面

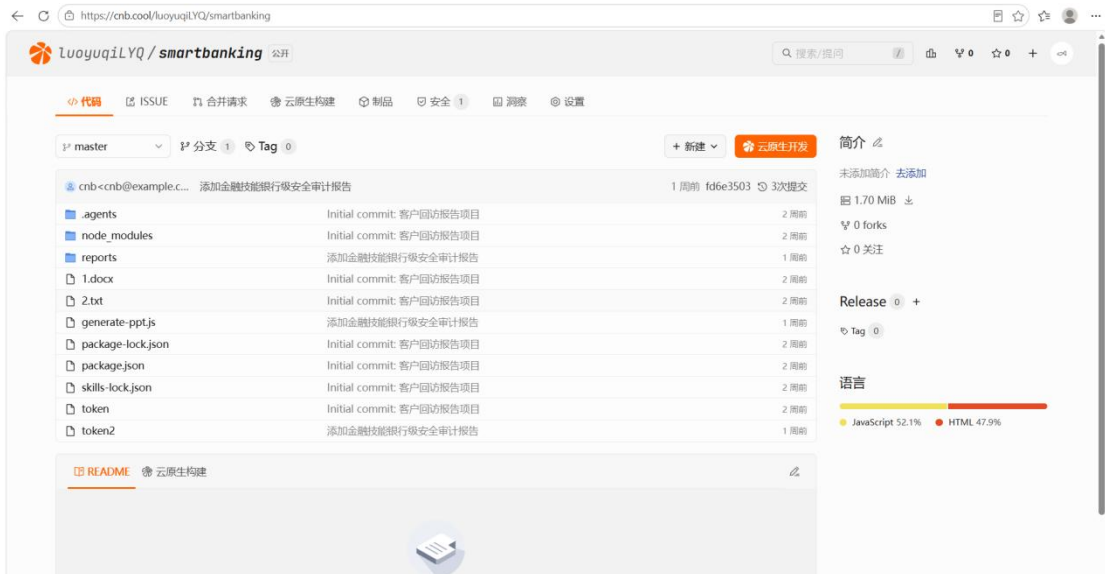
熟悉并配置了 BMAD，然后进行了一些熟悉操作

配置 git 用户信息，生成 SSH 公钥并添加到 CNB。

修改远程地址为 SSH 格式，成功 push 所有文件。

3、实验结果

成功把文件上传到 CNB 仓库了，并能够成功查看



3 branch 'master' set up to track 'origin/master'.

你的项目已经成功推送到 <https://cnb.cool/luoyuqiLYQ/smartbanking> 仓库!

上传详情

仓库地址: <https://cnb.cool/luoyuqiLYQ/smartbanking> 使用凭据: cnb CLI 的 access_token (已自动保存到 token2 文件) 推送状态: ☒ 成功

已上传的文件

1. ☒ reports/客户回访报告_C000001.docx - 客户电话回访报告
2. ☒ reports/credit_risk_hengda.md - 信用风险评估报告
3. ☒ reports/skill_audit_report.md - 金融技能银行级安全审计报告
4. ☒ 1.docx - 原始文档
5. ☒ 2.txt - 文本文件
6. ☒ token / token2 - 令牌文件
7. ☒ package.json / package-lock.json - 项目依赖
8. ☒ upload_to_cnb.js / upload_to_cnb.ps1 - 上传脚本

解决过程总结

6. ☒ token / token2 - 令牌文件
7. ☒ package.json / package-lock.json - 项目依赖
8. ☒ upload_to_cnb.js / upload_to_cnb.ps1 - 上传脚本

解决过程总结

1. cnb login 成功 - 获得了新的 access_token
2. 更新 token2 - 将新的 access_token 写入 token2 文件
3. 配置远程仓库 - 嵌入凭据到 URL 中
4. 成功推送 - 项目已同步到 cnb.cool

你现在可以访问 <https://cnb.cool/luoyuqiLYQ/smartbanking> 查看你的项目了! 🎉

把没有配置好的环境配置好了



4、心得

第三次实验对我来说，更像是一次“扫雷”行动。前两次留下的 CNB 上传问题一直卡在心里，这次终于把它彻底解决了。配置 SSH 密钥、修改远程地址、执行 `ssh -T` 测试，每一步都很顺利，当 `git push` 终于成功的那一刻，确实松了一口气。回头想想，之前失败的原因其实很简单——缺少正确的认证配置，但就因为这点小问题，整个流程就卡了两节课。这让我深刻体会到，开发环境里没有“差不多就行”的说法，一个小漏洞就能让后续所有工作停摆。

另外，接触 BMAD 框架也让我对软件开发有了新的认识。以前总觉得写代码就是噼里啪啦敲键盘，但 BMAD 强调的是先把需求、架构、设计想清楚，文档写明白，再动手开发。一开始觉得有点“啰嗦”，但仔细想想，做金融项目本来就要求严谨规范，先梳理清楚确实比边写边想要靠谱得多。

总的来说，这次实验让我意识到，用好 AI 工具的前提是把基础环境

彻底打通，而严谨的流程管理对金融类项目来说更是基本功。

实验四：小组作业（银行知识图谱构建与问答）

我们小组选择的是教程中给出的项目的第九项，即“银行知识图谱构建与问答基于银行产品、业务流程、监管法规等知识构建知识图谱，实现基于图谱的智能问答。技术栈：Python+Neo4j（或 JSON 模拟）+LLM，需设计本体和实体关系”，以下是我们的实验过程。

1、实验目的

理解知识图谱在金融领域（银行产品、流程、法规）的应用价值。

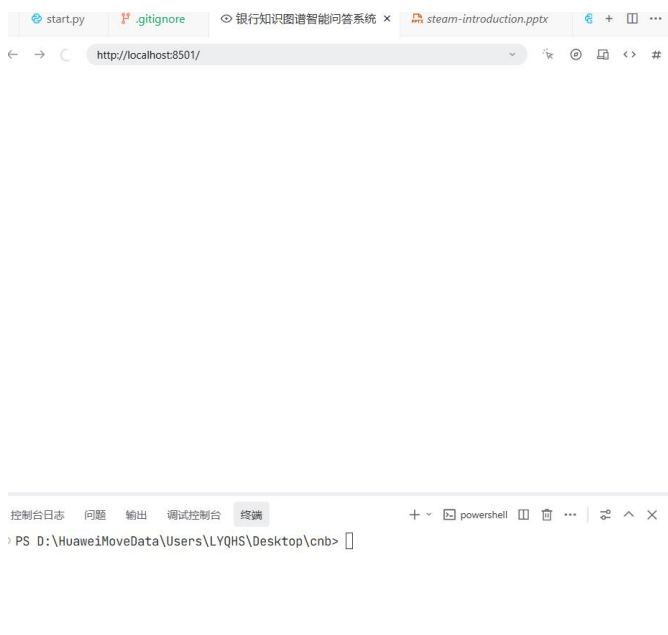
掌握本体设计、实体与关系抽取方法。

实现基于图谱的智能问答

2、实验步骤

第一次实验步骤是我先做的，我先草拟一个提纲然后提交，让 AI 进行代码生成，但是生成结果不如人意。虽说访问的界面显得很标致很完美很清晰，但是实际上是无法使用的，就是无论输入什么问题都得不到答案。

[银行知识图谱智能问答系统](#)

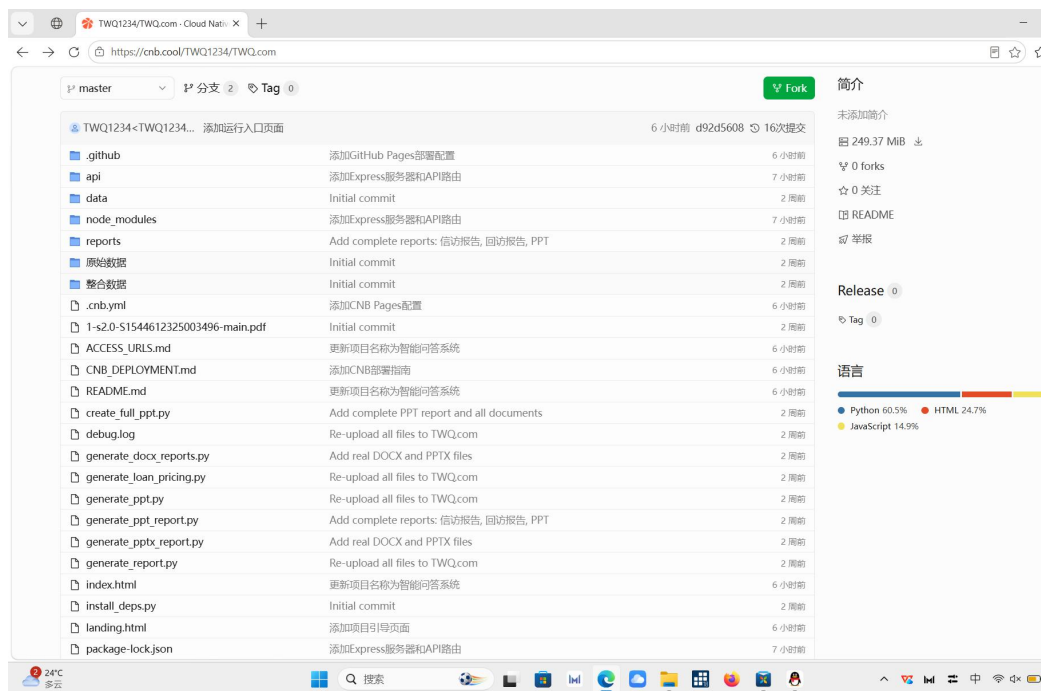


鉴于我的电脑无能为力，我的搭档进行了第二次尝试。她的代码运行很顺畅，并且功能运行完好，我根据她的实验改进了一下我的设计，但是不仅无法运行出结果，甚至连第一次实验的网页都打不开了，遂放弃。

以下是设计网页的使用方法：

TWQ1234/TWQ.com • Cloud Native Build,

登录该网址，选择 index，



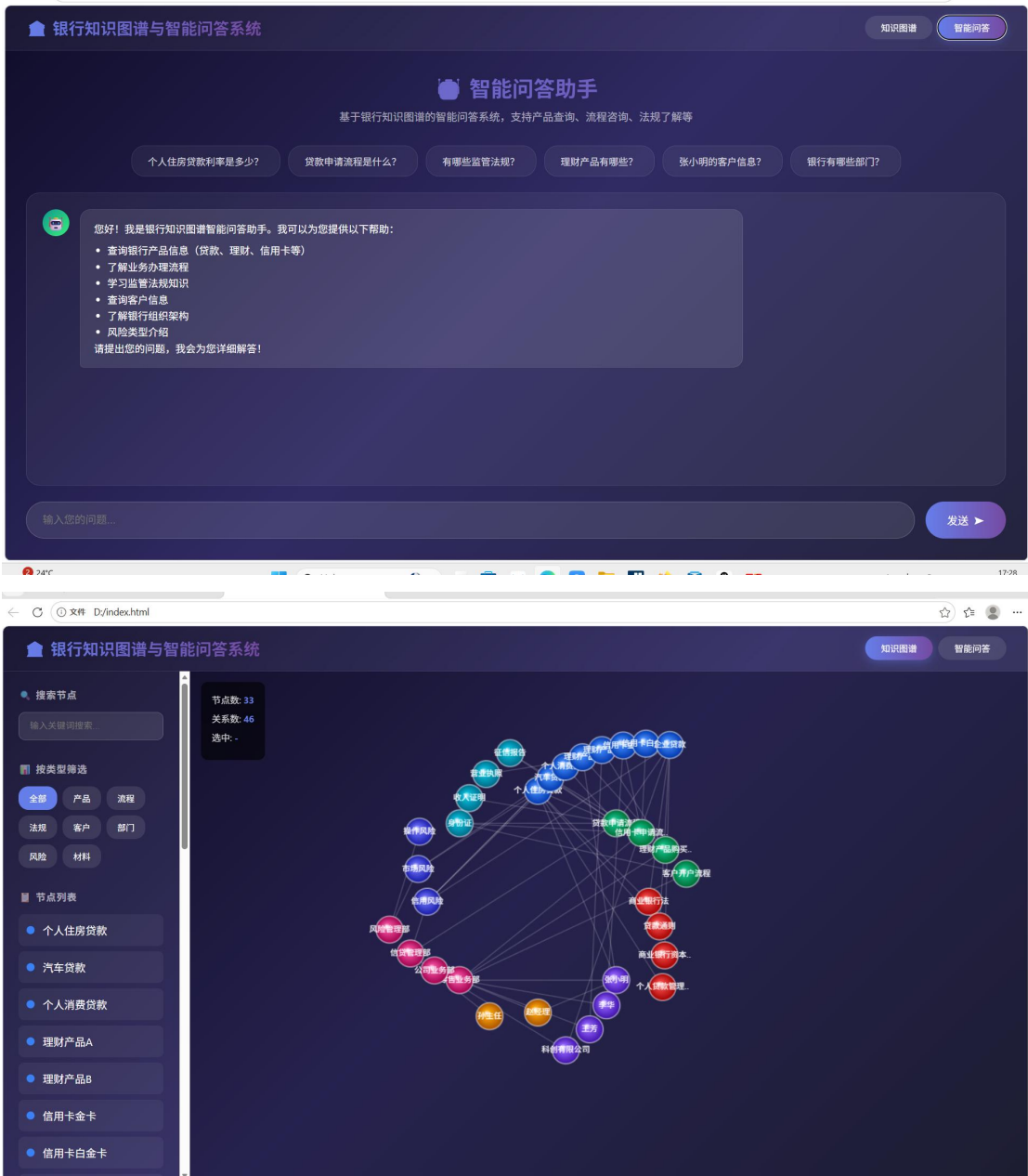
点击下载，



下载后点击打开文件即可使用



这是运行界面截图



3、实验结果

协作研究，共同生成了能够顺利运行的银行知识图谱构建与问答网站，并进行了简单问答

4、心得

这次知识图谱实验可以说是四次实验中最具挫败感的一次，但也让我对 AI 辅助开发有了更清醒的认知。

我的第一步其实挺顺利——用 AI 快速生成了前端问答界面，界面布局清晰、样式完整，看起来已经是一个可以交付的产品。但问题很快暴露出来：无论输入什么问题，系统都没有任何有效回复。排查后发现，AI 只完成了前端展示层的搭建，后端与知识图谱、LLM 的调用逻辑根本没有打通。界面再好看，也只是个空壳。

随后我的搭档进行了第二次尝试，她重新梳理了整体架构，确保前后端联动和接口调用的完整性，最终代码顺利运行。我参考她的设计思路重新调整自己的方案，结果不仅没修好，反而因为改动过大导致网页直接打不开，最终选择放弃。

这次经历给我最大的教训是：AI 生成代码时倾向于优先呈现“看得见”的部分，而最容易忽略的是底层逻辑的贯通。如果使用者不具备辨别能力，很容易被表面的完美所迷惑。同时我也意识到，在有限时间内及时止损、借鉴他人成果也是一种务实的选择。对工具保持清醒、对自己的局限保持诚实，或许比盲目坚持更有价值。